

Welcome to Day 20

Advanced OpenAI Agents SDK

TODAY'S TOPICS

- Multi-model orchestration with OpenAI Agents SDK
 - Structured outputs using Pydantic models
 - Input and output guardrails for safety
 - Building a production-ready Deep Research Agent
 - Converting notebooks to Python modules with Gradio UI
-

Recap from Day 19

What We Learned Previously



- **Tools:** Using function decorators to wrap Python functions as agent tools
- **Agents as Tools:** Converting agents into callable tools for other agents
- **Handoffs:** Transferring control between agents (vs. calling and returning)
- **Async IO:** Running multiple operations in parallel for efficiency



KEY DISTINCTION

Tools = call and return | **Handoffs = transfer control**

Today's Journey

Three Major Extensions



- **1. Multi-Model Orchestration:** Using models beyond OpenAI (Gemini, DeepSeek, Grok)
- **2. Structured Outputs:** Requiring agents to return specific data structures
- **3. Guardrails:** Protecting inputs and outputs with validation agents



HANDS-ON BUILD

Plus: Building a complete Deep Research Agent with proper Python modules!

Multi-Model Orchestration

Why Use Different Models?



Different models have different strengths:

- **Cost efficiency:** Some models are cheaper for simple tasks
- **Specialized capabilities:** Some excel at specific domains
- **Redundancy:** Multiple perspectives on the same problem
- **Avoiding vendor lock-in:** Flexibility to switch providers



OPENAI AGENTS SDK

Seamless integration through OpenAI-compatible endpoints

Setting Up Alternative Models

OpenAI-Compatible Endpoints

☰ SUPPORTED PROVIDERS

Many providers offer OpenAI-compatible APIs that work seamlessly with the SDK:

 Google Gemini

 DeepSeek

 Grok (xAI)

 Open Router

⚙️ CONFIGURATION & PROCESS

Required Configuration

- 1 Base URL**
Provider's specific API endpoint address
- 2 API Key**
Your unique authentication credential
- 3 Model Name**
Specific model identifier (e.g., "gemini-1.5-pro")

Implementation Process

```
</> PYTHON IMPLEMENTATION
```

```
client = AsyncOpenAI(base_url=..., api_key=...)
```

```
// Pass model object to Agent
```

```
agent = Agent(model="custom-model-name", client=client)
```


Multi-Model Example

Practical Implementation & Best Practices

🔗 PARALLEL EXECUTION SETUP

Example scenario: Three specialized sales agents running concurrently.

 **DeepSeek**
Sales Analytics

COST-EFFECTIVE

 **Gemini**
Copywriting

CREATIVE

 **Llama 3.3**
Lead Scoring

FAST

ASYNCIO PERFORMANCE

Total Time $\approx \text{Max}(t_1, t_2, t_3)$

Not the sum of all execution times

⚠️ CAUTIONARY TALE

∞ THE INFINITE LOOP RISK

Autonomous agents can get stuck in recursive loops, rapidly consuming tokens and budget without producing results.

Safety Protocols

- ✓ **Explicit Constraints**
Define strict boundaries for agent autonomy.
- ✓ **Hard Timeouts**
Set maximum execution time for every async call.
- ✓ **Iteration Limits**
Cap the number of retry attempts or tool calls (e.g., max 5).
- ✓ **Resource Monitoring**
Track token usage and costs in real-time.

Structured Outputs Overview

Beyond Text Responses

THE PROBLEM

LLMs naturally generate **unstructured text**, making programmatic use difficult and error-prone.



THE SOLUTION

Structured outputs force responses into **specific schemas** defined by code.

KEY BENEFITS

- Predictable data structures for downstream processing
- Easier integration with databases and APIs
- Type safety and validation
- Reduced parsing errors

Pydantic Models & Structured Outputs

Defining Data Schemas

PYDANTIC BASICS

Pydantic is Python's premier data validation library using Python type hints. It forces unstructured LLM responses into rigid, type-safe structures.

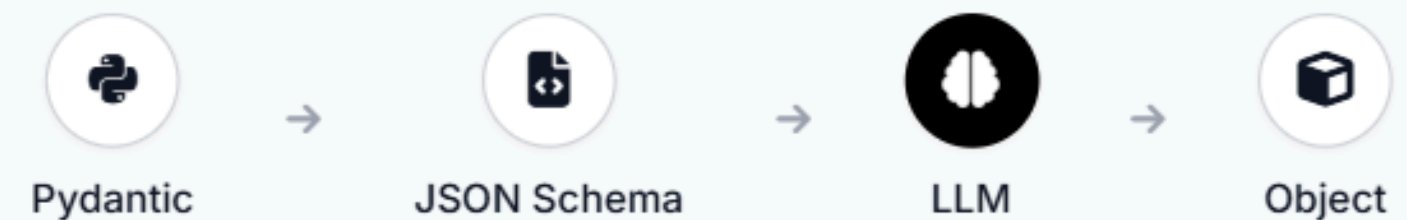
- ✓ Define classes inheriting from `BaseModel`
- ✓ Type-annotated fields with docstrings (visible to LLMs)
- ✓ Automatic validation, parsing, and serialization

`</>` SCHEMA DEFINITION

```
class ResearchResult(BaseModel):  
    summary: str  
    """Executive summary of findings"""  
    confidence: float  
    sources: List[str]
```

WORKFLOW & STRATEGY

DATA FLOW ARCHITECTURE



Pro Tip: The "Reason" Field

Always include a `reason` or `thought_process` field before your main data output.

WHY IT WORKS (CHAIN OF THOUGHT)

- Forces model to articulate reasoning first
- Significantly improves accuracy of subsequent fields

Guardrails for System Protection

Input & Output Validation

🛡️ VALIDATION CHECKPOINTS

Guardrails act as intelligent, LLM-powered safety gates at the edges of your agent system.



Input Guardrails

Validate incoming user requests **before** any processing occurs. Filters out malicious, irrelevant, or unsafe inputs immediately.



Output Guardrails

Validate the final response **before** showing it to users. Ensures quality, safety, and appropriate formatting.

🧠 WHY LLM-BASED?

Semantic Understanding Beyond Patterns

- ✓ **Detect PII & Sensitive Data:** Identifies names, addresses, and private info contextually.
- ✓ **Content Moderation:** Filters inappropriate, offensive, or outdated information.
- ✓ **Business Logic:** Validates tone, response completeness, and adherence to company policies.



Architectural Rule

Guardrails should only be placed at **system boundaries** (Input & Output). Do not place them mid-workflow between internal agents, as this creates unnecessary latency and complexity.

Implementing Guardrails

The Decorator Pattern



</> SYNTAX PATTERN

```
@input_guardrail
# OR
@output_guardrail

def
  validate_content(ctx, value):
```

RETURNS OBJECT STRUCTURE

GuardrailFunctionOutput

- 1 output_info : Metadata for system tracing
- 2 tripwire_triggered : Boolean flag

REAL-WORLD EXAMPLE

Name Check Agent detecting PII (Personally Identifiable Information) in user messages before processing.

TRIPWIRE WORKFLOW LOGIC



Tripwire Triggered

Validation logic returns `True`



Exception Raised

System immediately throws intervention exception



Workflow Stops

Current execution context is halted instantly



No Processing

Downstream agents/tools are never called

Deep Research Agent Overview

The Classic Agentic Use Case

AGENT WORKFLOW



Research Query & Planning

Takes user query and autonomously generates a structured search plan.



Parallel Execution

Executes multiple searches simultaneously using async IO for maximum speed.



Synthesis & Reporting

Aggregates findings into a coherent 1000+ word report and emails it to the user.

WHY BUILD YOUR OWN?



Full Customization

Tailor the research depth, focus areas, and tone specifically for your domain or business logic.



Cost & Depth Control

Manage budget by controlling search iterations and context size (unlike expensive SaaS wrappers).



Proprietary Data

Seamlessly integrate your own internal documents and databases into the research process.



Universal Applicability

A scalable architecture that works across industries—from market research to medical literature.

OpenAI Web Search Tool

Hosted Tool & Costs

HOSTED INFRASTRUCTURE

OpenAI provides managed tools that execute directly on their secure infrastructure:



Web Search Tool

Primary tool for Deep Research Agent. Enables real-time internet access and browsing.

SELECTED FOR PROJECT



File Search Tool

RAG capabilities for documents



Computer Tool

System interaction capabilities

PRICING & CONSTRAINTS

Pricing Model (Early 2025)

GPT-4o-mini

Single Search (Low Context)	\$0.025
3 Searches (Avg. Query)	≈ \$0.08
20 Searches (Deep Dive)	≈ \$0.50

⚠ Cost Warning

Extended research sessions can quickly accumulate costs. A single deep research session typically costs between **\$1.00 - \$3.00**.

Always implement usage caps and monitoring.

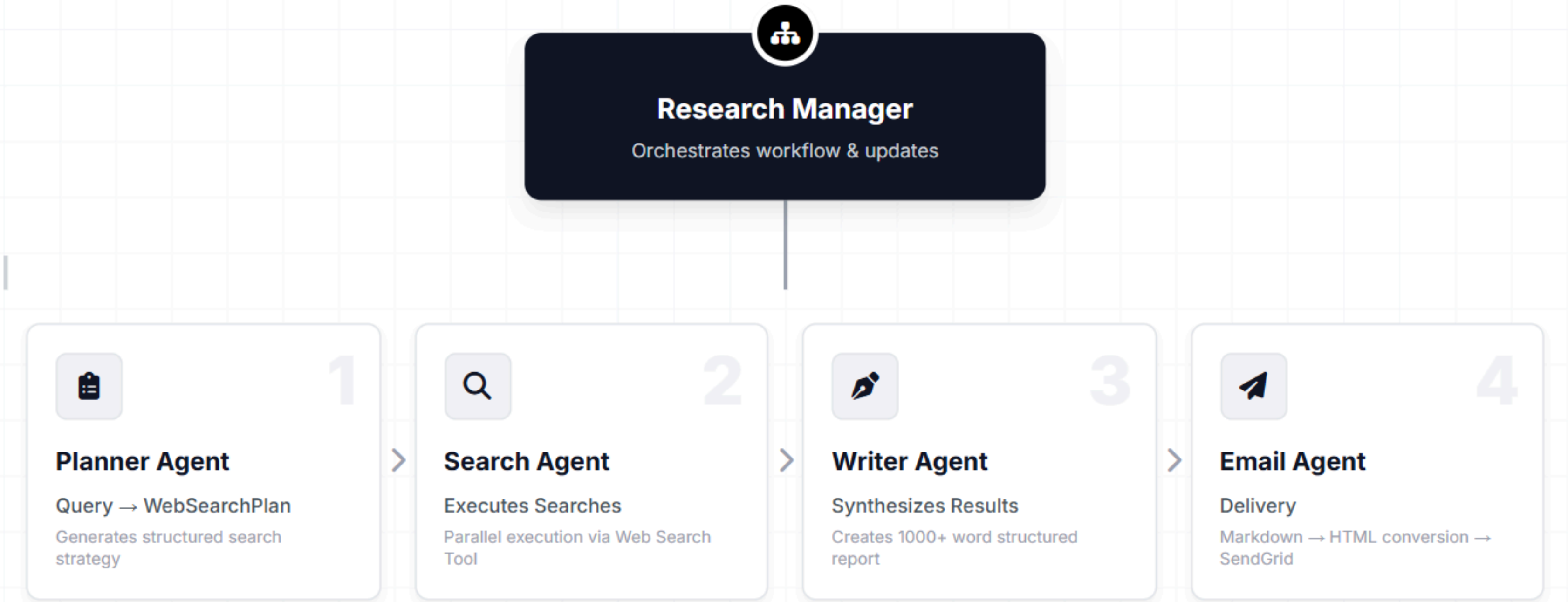
💡 Optimization Tip

Configure the tool to use **"low"** search context size. This significantly reduces token consumption while maintaining result relevance.

Deep Research Architecture

Five Key Components

AsyncIO Workflow



From Notebook to Production

Building a Web Interface



- **Transition Workflow:** Notebooks → Python Modules → Gradio UI
- **Gradio Benefits:** Requires no frontend knowledge to build interactive apps
- **Core Components:** Blocks, Textbox, Button, Markdown
- **Live Updates:** Generator callbacks provide real-time progress feedback
- **Deployment:** Easily deploy to Hugging Face Spaces for free hosting



GOAL

Your Deep Research Agent becomes a shareable portfolio piece!

Enhancements & Applications

Taking It Further

↗ ENHANCEMENT OPPORTUNITIES

- 1 Clarifying Questions**
Ask users for details before starting research to improve relevance.
- 2 Autonomous Manager**
Self-directed decision making on when to search deeper or stop.
- 3 Evaluator-Optimizer**
Quality control pattern to review and refine outputs automatically.
- 4 Query Refinement**
Dynamically adjust search queries based on initial findings.
- 5 Proprietary Data Sources**
Integrate internal databases or custom APIs alongside web search.

👛 REAL-WORLD APPLICATIONS



Market Intelligence

Competitor analysis and trend tracking



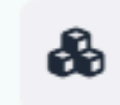
Legal & Technical

Case law research and documentation review



Medical Literature

Systematic reviews of research papers



Universal Architecture

Adaptable framework across any domain

"Your Deep Research Agent becomes a shareable portfolio piece!"

Key Takeaways & Resources

What We Learned Today

✓ SESSION SUMMARY

- ✓ **Multi-model orchestration**
Seamless integration with OpenAI-compatible endpoints
- ✓ **Structured outputs via Pydantic**
Ensuring predictable responses and type safety
- ✓ **Input & Output Guardrails**
Robust validation for system protection and safety
- ✓ **Production Deep Research Agent**
Complete architecture from planning to reporting
- ✓ **Gradio UI Deployment**
Converting notebooks to shareable web interfaces

📖 DOCUMENTATION & TOOLS



OpenAI Agents SDK

platform.openai.com/docs/agents



Gradio

gradio.app



Pydantic

docs.pydantic.dev



PRO TIP

Review these documentation sources to deeply understand the parameters available for each tool.

Challenge & Next Steps

Take It to the Next Level



YOUR CHALLENGE MISSION

- **Clarifying Questions:** Add logic to ask user for details before searching
- **Evaluator-Optimizer:** Implement a loop to quality-check and refine results
- **Deploy & Share:** Host your agent on Hugging Face Spaces

▶▶ **Coming Next:** Crew AI — A different agent framework focusing on role-based design



PORTFOLIO ACHIEVEMENT UNLOCKED

✅ Multi-agent SDR ✅ Career Agent ✅ Deep Research Agent

Keep experimenting and building! 🚀